IEModule4Assignment

October 22, 2026

1 Assignment 4: Transformers

This week, you learned about transformers. In this assignment, you will be implementing aspects of a transformer pipeline in python in order to again make predictions on fields of the address dataset.

[1]:
```
import torch
from torch.utils.data import Dataset, DataLoader
from sklearn.model_selection import train_test_split
from transformers import BertTokenizerFast, BertForTokenClassification, AdamW
import json
import numpy as np
no_deprecation_warning=True
# Load the dataset
with open("assets/list_of_real_usa_addresses.json", "r") as file:
    addresses_json = json.load(file)

# Define constants
PAD_TOKEN = "<PAD>"
UNK_TOKEN = "<UNK>"
```

2 Task 1: Set up data

In this task, you will be responsible for formatting the dataset into the arrays X and y, where X is a list of addresses split up into tokens, and y is a list containing the corresponding labels.

Ex. For the following data:

`[{ "address": "123 Main St", "city": "Springfield", "state": "IL", "zip": "62704" }, etc.]`

The function should create X and y like so:

`X = [["123", "Main", "St", "Springfield", "IL", "62704"], etc.]`

`y = [["address", "address", "address", "city", "state", "zip"], etc.]`

The solution for this code is the same as in assignment 3.

[2]:
```
task_id = '1'

## Utility Code will go here
```

[3]:
```
def task_1_solution():
    X = []
    y = []

    # YOUR CODE HERE
    #raise NotImplementedError()
    for address in addresses_json:
        Xi, yi = [], []
        for label, text in address.items():
            tokens = text.split()
            labels = [label for _ in tokens]
            Xi += tokens
            yi += labels
        X.append(Xi)
        y.append(yi)

    return X, y
```

[4]:
```
# Use this cell to explore your solution.

task_1_solution()
```

[4]:
```
([['777', 'Brockton', 'Avenue', 'Abington', 'MA', '2351'],
  ['30', 'Memorial', 'Drive', 'Avon', 'MA', '2322'],
  ['250', 'Hartford', 'Avenue', 'Bellingham', 'MA', '2019'],
  ['700', 'Oak', 'Street', 'Brockton', 'MA', '2301'],
  ['66-4', 'Parkhurst', 'Rd', 'Chelmsford', 'MA', '1824'],
  ['591', 'Memorial', 'Dr', 'Chicopee', 'MA', '1020'],
  ['55', 'Brooksby', 'Village', 'Way', 'Danvers', 'MA', '1923'],
  ['137', 'Teaticket', 'Hwy', 'East', 'Falmouth', 'MA', '2536'],
  ['42', 'Fairhaven', 'Commons', 'Way', 'Fairhaven', 'MA', '2719'],
  ['374', 'William', 'S', 'Canning', 'Blvd', 'Fall', 'River', 'MA', '2721'],
  ['121', 'Worcester', 'Rd', 'Framingham', 'MA', '1701'],
  ['677', 'Timpany', 'Blvd', 'Gardner', 'MA', '1440'],
  ['337', 'Russell', 'St', 'Hadley', 'MA', '1035'],
  ['295', 'Plymouth', 'Street', 'Halifax', 'MA', '2338'],
  ['1775', 'Washington', 'St', 'Hanover', 'MA', '2339'],
  ['280', 'Washington', 'Street', 'Hudson', 'MA', '1749'],
  ['20', 'Soojian', 'Dr', 'Leicester', 'MA', '1524'],
  ['11', 'Jungle', 'Road', 'Leominster', 'MA', '1453'],
  ['301', 'Massachusetts', 'Ave', 'Lunenburg', 'MA', '1462'],
  ['780', 'Lynnway', 'Lynn', 'MA', '1905'],
  ['70', 'Pleasant', 'Valley', 'Street', 'Methuen', 'MA', '1844'],
```

```
['830', 'Curran', 'Memorial', 'Hwy', 'North', 'Adams', 'MA', '1247'],
['1470', 'S', 'Washington', 'St', 'North', 'Attleboro', 'MA', '2760'],
['506', 'State', 'Road', 'North', 'Dartmouth', 'MA', '2747'],
['742', 'Main', 'Street', 'North', 'Oxford', 'MA', '1537'],
['72', 'Main', 'St', 'North', 'Reading', 'MA', '1864'],
['200', 'Otis', 'Street', 'Northborough', 'MA', '1532'],
['180', 'North', 'King', 'Street', 'Northhampton', 'MA', '1060'],
['555', 'East', 'Main', 'St', 'Orange', 'MA', '1364'],
['555', 'Hubbard', 'Ave-Suite', '12', 'Pittsfield', 'MA', '1201'],
['300', 'Colony', 'Place', 'Plymouth', 'MA', '2360'],
['301', 'Falls', 'Blvd', 'Quincy', 'MA', '2169'],
['36', 'Paramount', 'Drive', 'Raynham', 'MA', '2767'],
['450', 'Highland', 'Ave', 'Salem', 'MA', '1970'],
['1180', 'Fall', 'River', 'Avenue', 'Seekonk', 'MA', '2771'],
['1105', 'Boston', 'Road', 'Springfield', 'MA', '1119'],
['100', 'Charlton', 'Road', 'Sturbridge', 'MA', '1566'],
['262', 'Swansea', 'Mall', 'Dr', 'Swansea', 'MA', '2777'],
['333', 'Main', 'Street', 'Tewksbury', 'MA', '1876'],
['550', 'Providence', 'Hwy', 'Walpole', 'MA', '2081'],
['352', 'Palmer', 'Road', 'Ware', 'MA', '1082'],
['3005', 'Cranberry', 'Hwy', 'Rt', '6', '28', 'Wareham', 'MA', '2538'],
['250', 'Rt', '59', 'Airmont', 'NY', '10901'],
['141', 'Washington', 'Ave', 'Extension', 'Albany', 'NY', '12205'],
['13858', 'Rt', '31', 'W', 'Albion', 'NY', '14411'],
['2055', 'Niagara', 'Falls', 'Blvd', 'Amherst', 'NY', '14228'],
['101', 'Sanford', 'Farm', 'Shpg', 'Center', 'Amsterdam', 'NY', '12010'],
['297', 'Grant', 'Avenue', 'Auburn', 'NY', '13021'],
['4133', 'Veterans', 'Memorial', 'Drive', 'Batavia', 'NY', '14020'],
['6265', 'Brockport', 'Spencerport', 'Rd', 'Brockport', 'NY', '14420'],
['5399', 'W', 'Genesse', 'St', 'Camillus', 'NY', '13031'],
['3191', 'County', 'rd', '10', 'Canandaigua', 'NY', '14424'],
['30', 'Catskill', 'Catskill', 'NY', '12414'],
['161', 'Centereach', 'Mall', 'Centereach', 'NY', '11720'],
['3018', 'East', 'Ave', 'Central', 'Square', 'NY', '13036'],
['100', 'Thruway', 'Plaza', 'Cheektowaga', 'NY', '14225'],
['8064', 'Brewerton', 'Rd', 'Cicero', 'NY', '13039'],
['5033', 'Transit', 'Road', 'Clarence', 'NY', '14031'],
['3949', 'Route', '31', 'Clay', 'NY', '13041'],
['139', 'Merchant', 'Place', 'Cobleskill', 'NY', '12043'],
['85', 'Crooked', 'Hill', 'Road', 'Commack', 'NY', '11725'],
['872', 'Route', '13', 'Cortlandville', 'NY', '13045'],
['279', 'Troy', 'Road', 'East', 'Greenbush', 'NY', '12061'],
['2465', 'Hempstead', 'Turnpike', 'East', 'Meadow', 'NY', '11554'],
['6438', 'Basile', 'Rowe', 'East', 'Syracuse', 'NY', '13057'],
['25737', 'US', 'Rt', '11', 'Evans', 'Mills', 'NY', '13637'],
['901', 'Route', '110', 'Farmingdale', 'NY', '11735'],
['2400', 'Route', '9', 'Fishkill', 'NY', '12524'],
```

```
['10401', 'Bennett', 'Road', 'Fredonia', 'NY', '14063'],
['1818', 'State', 'Route', '3', 'Fulton', 'NY', '13069'],
['4300', 'Lakeville', 'Road', 'Geneseo', 'NY', '14454'],
['990', 'Route', '5', '20', 'Geneva', 'NY', '14456'],
['311', 'RT', '9W', 'Glenmont', 'NY', '12077'],
['200', 'Dutch', 'Meadows', 'Ln', 'Glenville', 'NY', '12302'],
['100', 'Elm', 'Ridge', 'Center', 'Dr', 'Greece', 'NY', '14626'],
['1549', 'Rt', '9', 'Halfmoon', 'NY', '12065'],
['5360', 'Southwestern', 'Blvd', 'Hamburg', 'NY', '14075'],
['103', 'North', 'Caroline', 'St', 'Herkimer', 'NY', '13350'],
['1000', 'State', 'Route', '36', 'Hornell', 'NY', '14843'],
['1400', 'County', 'Rd', '64', 'Horseheads', 'NY', '14845'],
['135', 'Fairgrounds', 'Memorial', 'Pkwy', 'Ithaca', 'NY', '14850'],
['2', 'Gannett', 'Dr', 'Johnson', 'City', 'NY', '13790'],
['233', '5th', 'Ave', 'Ext', 'Johnstown', 'NY', '12095'],
['601', 'Frank', 'Stottile', 'Blvd', 'Kingston', 'NY', '12401'],
['350', 'E', 'Fairmount', 'Ave', 'Lakewood', 'NY', '14750'],
['4975', 'Transit', 'Rd', 'Lancaster', 'NY', '14086'],
['579', 'Troy-Schenectady', 'Road', 'Latham', 'NY', '12110'],
['5783', 'So', 'Transit', 'Road', 'Lockport', 'NY', '14094'],
['7155', 'State', 'Rt', '12', 'S', 'Lowville', 'NY', '13367'],
['425', 'Route', '31', 'Macedon', 'NY', '14502'],
['3222', 'State', 'Rt', '11', 'Malone', 'NY', '12953'],
['200', 'Sunrise', 'Mall', 'Massapequa', 'NY', '11758'],
['43', 'Stephenville', 'St', 'Massena', 'NY', '13662'],
['750', 'Middle', 'Country', 'Road', 'Middle', 'Island', 'NY', '11953'],
['470', 'Route', '211', 'East', 'Middletown', 'NY', '10940'],
['3133', 'E', 'Main', 'St', 'Mohegan', 'Lake', 'NY', '10547'],
['288', 'Larkin', 'Monroe', 'NY', '10950'],
['41', 'Anawana', 'Lake', 'Road', 'Monticello', 'NY', '12701'],
['4765', 'Commercial', 'Drive', 'New', 'Hartford', 'NY', '13413'],
['1201', 'Rt', '300', 'Newburgh', 'NY', '12550'],
['255', 'W', 'Main', 'St', 'Avon', 'CT', '6001'],
['120', 'Commercial', 'Parkway', 'Branford', 'CT', '6405'],
['1400', 'Farmington', 'Ave', 'Bristol', 'CT', '6010'],
['161', 'Berlin', 'Road', 'Cromwell', 'CT', '6416'],
['67', 'Newton', 'Rd', 'Danbury', 'CT', '6810'],
['656', 'New', 'Haven', 'Ave', 'Derby', 'CT', '6418'],
['69', 'Prospect', 'Hill', 'Road', 'East', 'Windsor', 'CT', '6088'],
['150', 'Gold', 'Star', 'Hwy', 'Groton', 'CT', '6340'],
['900', 'Boston', 'Post', 'Road', 'Guilford', 'CT', '6437'],
['2300', 'Dixwell', 'Ave', 'Hamden', 'CT', '6514'],
['495', 'Flatbush', 'Ave', 'Hartford', 'CT', '6106'],
['180', 'River', 'Rd', 'Lisbon', 'CT', '6351'],
['420', 'Buckland', 'Hills', 'Dr', 'Manchester', 'CT', '6040'],
['1365', 'Boston', 'Post', 'Road', 'Milford', 'CT', '6460'],
['1100', 'New', 'Haven', 'Road', 'Naugatuck', 'CT', '6770'],
```

```
['315', 'Foxon', 'Blvd', 'New', 'Haven', 'CT', '6513'],
['164', 'Danbury', 'Rd', 'New', 'Milford', 'CT', '6776'],
['3164', 'Berlin', 'Turnpike', 'Newington', 'CT', '6111'],
['474', 'Boston', 'Post', 'Road', 'North', 'Windham', 'CT', '6256'],
['650', 'Main', 'Ave', 'Norwalk', 'CT', '6851'],
['680', 'Connecticut', 'Avenue', 'Norwalk', 'CT', '6854'],
['220', 'Salem', 'Turnpike', 'Norwich', 'CT', '6360'],
['655', 'Boston', 'Post', 'Rd', 'Old', 'Saybrook', 'CT', '6475'],
['625', 'School', 'Street', 'Putnam', 'CT', '6260'],
['80', 'Town', 'Line', 'Rd', 'Rocky', 'Hill', 'CT', '6067'],
['465', 'Bridgeport', 'Avenue', 'Shelton', 'CT', '6484'],
['235', 'Queen', 'St', 'Southington', 'CT', '6489'],
['150', 'Barnum', 'Avenue', 'Cutoff', 'Stratford', 'CT', '6614'],
['970', 'Torringford', 'Street', 'Torrington', 'CT', '6790'],
['844', 'No', 'Colony', 'Road', 'Wallingford', 'CT', '6492'],
['910', 'Wolcott', 'St', 'Waterbury', 'CT', '6705'],
['155', 'Waterford', 'Parkway', 'No', 'Waterford', 'CT', '6385'],
['515', 'Sawmill', 'Road', 'West', 'Haven', 'CT', '6516'],
['2473', 'Hackworth', 'Road', 'Adamsville', 'AL', '35005'],
['630', 'Coonial', 'Promenade', 'Pkwy', 'Alabaster', 'AL', '35007'],
['2643', 'Hwy', '280', 'West', 'Alexander', 'City', 'AL', '35010'],
['540', 'West', 'Bypass', 'Andalusia', 'AL', '36420'],
['5560', 'Mcclellan', 'Blvd', 'Anniston', 'AL', '36206'],
['1450', 'No', 'Brindlee', 'Mtn', 'Pkwy', 'Arab', 'AL', '35016'],
['1011', 'US', 'Hwy', '72', 'East', 'Athens', 'AL', '35611'],
['973', 'Gilbert', 'Ferry', 'Road', 'Se', 'Attalla', 'AL', '35954'],
['1717', 'South', 'College', 'Street', 'Auburn', 'AL', '36830'],
['701', 'Mcmeans', 'Ave', 'Bay', 'Minette', 'AL', '36507'],
['750', 'Academy', 'Drive', 'Bessemer', 'AL', '35022'],
['312', 'Palisades', 'Blvd', 'Birmingham', 'AL', '35209'],
['1600', 'Montclair', 'Rd', 'Birmingham', 'AL', '35210'],
['5919', 'Trussville', 'Crossings', 'Pkwy', 'Birmingham', 'AL', '35235'],
['9248', 'Parkway', 'East', 'Birmingham', 'AL', '35206'],
['1972', 'Hwy', '431', 'Boaz', 'AL', '35957'],
['10675', 'Hwy', '5', 'Brent', 'AL', '35034'],
['2041', 'Douglas', 'Avenue', 'Brewton', 'AL', '36426'],
['5100', 'Hwy', '31', 'Calera', 'AL', '35040'],
['1916', 'Center', 'Point', 'Rd', 'Center', 'Point', 'AL', '35215'],
['1950', 'W', 'Main', 'St', 'Centre', 'AL', '35960'],
['16077', 'Highway', '280', 'Chelsea', 'AL', '35043'],
['1415', '7Th', 'Street', 'South', 'Clanton', 'AL', '35045'],
['626', 'Olive', 'Street', 'Sw', 'Cullman', 'AL', '35055'],
['27520', 'Hwy', '98', 'Daphne', 'AL', '36526'],
['2800', 'Spring', 'Avn', 'SW', 'Decatur', 'AL', '35603'],
['969', 'Us', 'Hwy', '80', 'West', 'Demopolis', 'AL', '36732'],
['3300', 'South', 'Oates', 'Street', 'Dothan', 'AL', '36301'],
['4310', 'Montgomery', 'Hwy', 'Dothan', 'AL', '36303'],
```

```
['600', 'Boll', 'Weevil', 'Circle', 'Enterprise', 'AL', '36330'],
['3176', 'South', 'Eufaula', 'Avenue', 'Eufaula', 'AL', '36027'],
['7100', 'Aaron', 'Aronov', 'Drive', 'Fairfield', 'AL', '35064'],
['10040', 'County', 'Road', '48', 'Fairhope', 'AL', '36533'],
['3186', 'Hwy', '171', 'North', 'Fayette', 'AL', '35555'],
['3100', 'Hough', 'Rd', 'Florence', 'AL', '35630'],
['2200', 'South', 'Mckenzie', 'St', 'Foley', 'AL', '36535'],
['2001', 'Glenn', 'Bldv', 'Sw', 'Fort', 'Payne', 'AL', '35968'],
['340', 'East', 'Meighan', 'Blvd', 'Gadsden', 'AL', '35903'],
['890', 'Odum', 'Road', 'Gardendale', 'AL', '35071'],
['1608', 'W', 'Magnolia', 'Ave', 'Geneva', 'AL', '36340'],
['501', 'Willow', 'Lane', 'Greenville', 'AL', '36037'],
['170', 'Fort', 'Morgan', 'Road', 'Gulf', 'Shores', 'AL', '36542'],
['11697', 'US', 'Hwy', '431', 'Guntersville', 'AL', '35976'],
['42417', 'Hwy', '195', 'Haleyville', 'AL', '35565'],
['1706', 'Military', 'Street', 'South', 'Hamilton', 'AL', '35570'],
['1201', 'Hwy', '31', 'NW', 'Hartselle', 'AL', '35640'],
['209', 'Lakeshore', 'Parkway', 'Homewood', 'AL', '35209'],
['2780', 'John', 'Hawkins', 'Pkwy', 'Hoover', 'AL', '35244'],
['5335', 'Hwy', '280', 'South', 'Hoover', 'AL', '35242'],
['1007', 'Red', 'Farmer', 'Drive', 'Hueytown', 'AL', '35023'],
['2900', 'S', 'Mem', 'PkwyDrake', 'Ave', 'Huntsville', 'AL', '35801'],
['11610', 'Memorial', 'Pkwy', 'South', 'Huntsville', 'AL', '35803'],
['2200', 'Sparkman', 'Drive', 'Huntsville', 'AL', '35810'],
['330', 'Sutton', 'Rd', 'Huntsville', 'AL', '35763'],
['6140A', 'Univ', 'Drive', 'Huntsville', 'AL', '35806'],
['4206', 'N', 'College', 'Ave', 'Jackson', 'AL', '36545'],
['1625', 'Pelham', 'South', 'Jacksonville', 'AL', '36265'],
['1801', 'Hwy', '78', 'East', 'Jasper', 'AL', '35501'],
['8551', 'Whitfield', 'Ave', 'Leeds', 'AL', '35094'],
['8650', 'Madison', 'Blvd', 'Madison', 'AL', '35758'],
['145', 'Kelley', 'Blvd', 'Millbrook', 'AL', '36054'],
['1970', 'S', 'University', 'Blvd', 'Mobile', 'AL', '36609'],
['6350', 'Cottage', 'Hill', 'Road', 'Mobile', 'AL', '36609'],
['101', 'South', 'Beltline', 'Highway', 'Mobile', 'AL', '36606'],
['2500', 'Dawes', 'Road', 'Mobile', 'AL', '36695'],
['5245', 'Rangeline', 'Service', 'Rd', 'Mobile', 'AL', '36619'],
['685', 'Schillinger', 'Rd', 'Mobile', 'AL', '36695'],
['3371', 'S', 'Alabama', 'Ave', 'Monroeville', 'AL', '36460'],
['10710', 'Chantilly', 'Pkwy', 'Montgomery', 'AL', '36117'],
['3801', 'Eastern', 'Blvd', 'Montgomery', 'AL', '36116'],
['6495', 'Atlanta', 'Hwy', 'Montgomery', 'AL', '36117'],
['851', 'Ann', 'St', 'Montgomery', 'AL', '36107'],
['15445', 'Highway', '24', 'Moulton', 'AL', '35650'],
['517', 'West', 'Avalon', 'Ave', 'Muscle', 'Shoals', 'AL', '35661'],
['5710', 'Mcfarland', 'Blvd', 'Northport', 'AL', '35476'],
['2453', '2Nd', 'Avenue', 'East', 'Oneonta', 'AL', '35121'],
```

```
 ['2900', 'Pepperrell', 'Pkwy', 'Opelika', 'AL', '36801'],
 ['92', 'Plaza', 'Lane', 'Oxford', 'AL', '36203'],
 ['1537', 'Hwy', '231', 'South', 'Ozark', 'AL', '36360'],
 ['2181', 'Pelham', 'Pkwy', 'Pelham', 'AL', '35124'],
 ['165', 'Vaughan', 'Ln', 'Pell', 'City', 'AL', '35125'],
 ['3700', 'Hwy', '280-431', 'N', 'Phenix', 'City', 'AL', '36867'],
 ['1903', 'Cobbs', 'Ford', 'Rd', 'Prattville', 'AL', '36066'],
 ['4180', 'Us', 'Hwy', '431', 'Roanoke', 'AL', '36274'],
 ['13675', 'Hwy', '43', 'Russellville', 'AL', '35653'],
 ['1095', 'Industrial', 'Pkwy', 'Saraland', 'AL', '36571'],
 ['24833', 'Johnt', 'Reidprkw', 'Scottsboro', 'AL', '35768'],
 ['1501', 'Hwy', '14', 'East', 'Selma', 'AL', '36703'],
 ['7855', 'Moffett', 'Rd', 'Semmes', 'AL', '36575'],
 ['150', 'Springville', 'Station', 'Blvd', 'Springville', 'AL', '35146'],
 ['690', 'Hwy', '78', 'Sumiton', 'AL', '35148'],
 ['41301', 'US', 'Hwy', '280', 'Sylacauga', 'AL', '35150'],
 ['214', 'Haynes', 'Street', 'Talladega', 'AL', '35160'],
 ['1300', 'Gilmer', 'Ave', 'Tallassee', 'AL', '36078'],
 ['34301', 'Hwy', '43', 'Thomasville', 'AL', '36784'],
 ['1420', 'Us', '231', 'South', 'Troy', 'AL', '36081'],
 ['1501', 'Skyland', 'Blvd', 'E', 'Tuscaloosa', 'AL', '35405'],
 ['3501', '20th', 'Av', 'Valley', 'AL', '36854'],
 ['1300', 'Montgomery', 'Highway', 'Vestavia', 'Hills', 'AL', '35216'],
 ['4538', 'Us', 'Hwy', '231', 'Wetumpka', 'AL', '36092'],
 ['2575', 'Us', 'Hwy', '43', 'Winfield', 'AL', '35594']],
[['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
 ['address',
  'address',
  'address',
  'address',
  'address',
  'city',
  'city',
  'state',
  'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
 ['address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
```

```
 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address',
 'address',
 'address',
 'address',
 'address',
 'city',
 'state',
 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
```

```
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'state', 'zip'],
['address', 'address', 'address', 'city', 'city', 'state', 'zip'],
```

```
    ['address', 'address', 'address', 'address', 'city', 'state', 'zip'],
    ['address', 'address', 'address', 'address', 'city', 'state', 'zip']])
```

[8]:
```
print(f"Task {task_id} - AG tests")

# Call the student solution
stu_X, stu_y = task_1_solution()

print(f"Task {task_id} - your X:\n{stu_X[:2]}")  # Display a subset of the data␣
 ↪for clarity
print(f"Task {task_id} - your y:\n{stu_y[:2]}")  # Display a subset of the data␣
 ↪for clarity

assert isinstance(stu_X, list), f"Task {task_id}: The returned X should be a␣
 ↪list."
assert isinstance(stu_y, list), f"Task {task_id}: The returned y should be a␣
 ↪list."

assert len(stu_X) == len(
    stu_y
), f"Task {task_id}: The lengths of X and y should match. Found␣
 ↪len(X)={len(stu_X)} and len(y)={len(stu_y)}."

assert all(
    isinstance(seq, list) and all(isinstance(word, str) for word in seq)
    for seq in stu_X
), f"Task {task_id}: Each element of X should be a list of strings."

assert all(
    isinstance(seq, list) and all(isinstance(label, str) for label in seq)
    for seq in stu_y
), f"Task {task_id}: Each element of y should be a list of strings."
```

```
Task 2 - AG tests
Task 2 - your X:
[['777', 'Brockton', 'Avenue', 'Abington', 'MA', '2351'], ['30', 'Memorial',
'Drive', 'Avon', 'MA', '2322']]
Task 2 - your y:
[['address', 'address', 'address', 'city', 'state', 'zip'], ['address',
'address', 'address', 'city', 'state', 'zip']]
```

3 Task 2: Encode Data

Like with our bidirectional LSTM CRF model, we must encode the data we've read into a format appropriate for our transformer model. You will be implementing a part of this.

The first step has been done for you, loading the tokenizer.

Then, we loop through in the `sequences` and `label` arrays in parallel.

For each token sequence and label sequence: - We define a `tokenized` dictionary with the current sequence as the first argument. We also use `is_split_into_words=True, truncation=True, padding="max_length", max_length=128, return_tensors="pt"`

- Now, we should be able to access elements of the `tokenized` dictionary with `[]` and access the "input_ids" and "attention_mask" fields. We call `.squeeze()` on the arrays we obtain when subscripting into the dictionary and append them to `input_ids` and `attention_masks`.
- The next step is to define label_ids. We create a new label id list variable that we will later append to `label_ids`
- This next part is where you come in!
- Still inside of the loop for each sequence and label, we write another loop that loops through the word ids in `tokenized.word_ids()` in the `word_ids` variable, which looks like this:

`[None, 0, 0, 1, 1, 2, 3, 3, 4, 5, 5, None, None, None,.... ]`

- Inside of this subloop:
  - Check each word id. If it is of type `None`, append the integer -100 to the `label_id_seq` list variable created before this subloop began.
  - Otherwise it should be an integer you can use to subscript into the current label sequence(`label_seq`) from the outer loop to obtain a label. Use the `state_to_idx` to convert that label into it's corresponding index. Append that to `label_id_seq`.
- Now, we exit the subloop and append `torch.tensor(('label_id_seq'), dtype=torch.long)` to label_ids. All done!

[5]:
```
task_id = '2'
```

[22]:
```
def task_2_solution():
    sequences, labels = task_1_solution()
    state_to_idx = {"address": 0, "city": 1, "state": 2, "zip": 3}


    tokenizer = BertTokenizerFast.from_pretrained("assets/tokenizer_dir")

    input_ids, attention_masks, label_ids = [], [], []


    for seq, label_seq in zip(sequences, labels):
        tokenized = tokenizer(seq, is_split_into_words=True, truncation=True,␣
↪padding="max_length", max_length=128, return_tensors="pt")

        input_ids.append(tokenized["input_ids"].squeeze())

        attention_masks.append(tokenized["attention_mask"].squeeze())

        word_ids = tokenized.word_ids()
```

```
        label_id_seq = []
        # YOUR CODE HERE
        #raise NotImplementedError()
        #print(word_ids)
        #print(label_seq)
        for word_id in word_ids:
            if word_id is None:
                label_id_seq.append(-100)
            else:
                label_id_seq.append(state_to_idx[label_seq[word_id]])

        label_ids.append(torch.tensor(label_id_seq, dtype=torch.long))

    return torch.stack(input_ids), torch.stack(attention_masks), torch.
↪stack(label_ids)
```

[23]:
```
# Use this cell to explore your solution.

task_2_solution()
```

[23]:
```
(tensor([[  101,  6255,  2581,  …,     0,     0,     0],
         [  101,  2382,  3986,  …,     0,     0,     0],
         [  101,  5539, 13381,  …,     0,     0,     0],
         …,
         [  101, 19527,  8482,  …,     0,     0,     0],
         [  101,  3429, 22025,  …,     0,     0,     0],
         [  101, 24368,  2629,  …,     0,     0,     0]]),
 tensor([[1, 1, 1,  …, 0, 0, 0],
         [1, 1, 1,  …, 0, 0, 0],
         [1, 1, 1,  …, 0, 0, 0],
         …,
         [1, 1, 1,  …, 0, 0, 0],
         [1, 1, 1,  …, 0, 0, 0],
         [1, 1, 1,  …, 0, 0, 0]]),
 tensor([[-100,    0,    0,  …, -100, -100, -100],
         [-100,    0,    0,  …, -100, -100, -100],
         [-100,    0,    0,  …, -100, -100, -100],
         …,
         [-100,    0,    0,  …, -100, -100, -100],
         [-100,    0,    0,  …, -100, -100, -100],
         [-100,    0,    0,  …, -100, -100, -100]]))
```

[24]:
```
print(f"Task {task_id} - AG tests")

stu_X, stu_attention_masks, stu_labels = task_2_solution()
```

```
print(f"Task {task_id} - your input_ids:\n{stu_X[:2]}")  # Display a subset of␣
 ↪the data for clarity
print(f"Task {task_id} - your attention_masks:\n{stu_attention_masks[:2]}")  #␣
 ↪Display a subset of the data for clarity
print(f"Task {task_id} - your labels:\n{stu_labels[:2]}")  # Display a subset␣
 ↪of the data for clarity

assert isinstance(stu_X, torch.Tensor), f"Task {task_id}: The returned `X`␣
 ↪should be a torch.Tensor."
assert isinstance(stu_attention_masks, torch.Tensor), f"Task {task_id}: The␣
 ↪returned `attention_masks` should be a torch.Tensor."
assert isinstance(stu_labels, torch.Tensor), f"Task {task_id}: The returned␣
 ↪`labels` should be a torch.Tensor."

assert stu_X.shape == stu_attention_masks.shape == stu_labels.shape, f"Task␣
 ↪{task_id}: The shapes of `X`, `attention_masks`, and `labels` should match.␣
 ↪Found X.shape={stu_X.shape}, attention_masks.shape={stu_attention_masks.
 ↪shape}, labels.shape={stu_labels.shape}"

assert stu_X.size(1) == 128, f"Task {task_id}: Each sequence in `X` should have␣
 ↪128 tokens. Found a sequence of length {stu_X.size(1)}."
assert stu_attention_masks.size(1) == 128, f"Task {task_id}: Each sequence in␣
 ↪`attention_masks` should have 128 tokens. Found a sequence of length␣
 ↪{stu_attention_masks.size(1)}."
assert stu_labels.size(1) == 128, f"Task {task_id}: Each sequence in `labels`␣
 ↪should have 128 tokens. Found a sequence of length {stu_labels.size(1)}."
```

```
Task 2 - AG tests
Task 2 - your input_ids:
tensor([[  101,  6255,  2581, 13899,  2669,  3927, 11113,  7853,  5003, 17825,
          2487,   102,     0,     0,     0,     0,     0,     0,     0,     0,
             0,     0
```

```
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0,     0,     0,
            0,     0,     0,     0,     0,     0,     0,     0]])
Task 2 - your attention_masks:
tensor([[1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0],
        [1, 1, 1, 1, 1, 1, 1, 1, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
         0, 0, 0, 0, 0, 0, 0, 0]])
Task 2 - your labels:
tensor([[-100,    0,    0,    0,    0,    0,    1,    1,    2,    3,    3, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100],
        [-100,    0,    0,    0,    1,    2,    3,    3, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100, -100,
         -100, -100, -100, -100, -100, -100, -100, -100]])
```

4 Task 3: Create Dataset class

Just like with Bidirectional LSTM CRF it helps us to create a dataset class so we can create Dataloaders, which store our data in batches.

Again, your job is to just define the `__init__` class, where you set `self.input_ids`, `self.attention_masks`, and `self.label_ids` to their corresponding passed in arguments.

Afterwards, create train and test `AddressDataset` objects from the variables we retrieved from `train_test_split`. For `train_dataset`, `input_ids` should be set to `X_train`, `attention_masks` to `attn_train`, and `label_ids` to `y_train`. You will do the same for `test_dataset` but with the test versions of these variables we have at the beginning.

Finally, create a train loader and test loader object with `Dataloader`, where the `dataset` arguments are the train and test datasets you just defined. Also set `batch_size` to 16.

The return statement has been defined for you.

[ ]:
```
task_id = '3'

## Utility Code will go here
```

[26]:
```
def task_3_solution():
    input_ids, attention_masks, label_ids = task_2_solution()


    X_train, X_test, attn_train, attn_test, y_train, y_test =␣
 ↪train_test_split(input_ids, attention_masks, label_ids, test_size=0.2,␣
 ↪random_state=42)

    # Define Dataset class
    class AddressDataset(Dataset):
        def __init__(self, input_ids, attention_masks, label_ids):
            # YOUR CODE HERE
            #raise NotImplementedError()
            self.input_ids = input_ids
            self.attention_masks = attention_masks
            self.label_ids = label_ids

        def __len__(self):
            return len(self.input_ids)

        def __getitem__(self, idx):
            return self.input_ids[idx], self.attention_masks[idx], self.
 ↪label_ids[idx]

    # Create datasets and loaders

    # YOUR CODE HERE
```

```
    #raise NotImplementedError()

    train_dataset = AddressDataset(X_train, attn_train, y_train)
    test_dataset = AddressDataset(X_test, attn_test, y_test)

    train_loader = torch.utils.data.DataLoader(train_dataset, batch_size=16,␣
 ↪shuffle=False)
    test_loader = torch.utils.data.DataLoader(test_dataset, batch_size=16)


    return train_loader, test_loader, input_ids, attention_masks, label_ids
```

[27]:
```
# Use this cell to explore your solution.

task_3_solution()
```

[27]:
```
(<torch.utils.data.dataloader.DataLoader at 0x75e1d99e79d0>,
 <torch.utils.data.dataloader.DataLoader at 0x75e1d99e4970>,
 tensor([[  101,  6255,  2581,  …,     0,     0,     0],
         [  101,  2382,  3986,  …,     0,     0,     0],
         [  101,  5539, 13381,  …,     0,     0,     0],
         …,
         [  101, 19527,  8482,  …,     0,     0,     0],
         [  101,  3429, 22025,  …,     0,     0,     0],
         [  101, 24368,  2629,  …,     0,     0,     0]]),
 tensor([[1, 1, 1,  …, 0, 0, 0],
         [1, 1, 1,  …, 0, 0, 0],
         [1, 1, 1,  …, 0, 0, 0],
         …,
         [1, 1, 1,  …, 0, 0, 0],
         [1, 1, 1,  …, 0, 0, 0],
         [1, 1, 1,  …, 0, 0, 0]]),
 tensor([[-100,    0,    0,  …, -100, -100, -100],
         [-100,    0,    0,  …, -100, -100, -100],
         [-100,    0,    0,  …, -100, -100, -100],
         …,
         [-100,    0,    0,  …, -100, -100, -100],
         [-100,    0,    0,  …, -100, -100, -100],
         [-100,    0,    0,  …, -100, -100, -100]]))
```

[28]:
```
print(f"Task {task_id} - AG tests")

# Call the student solution
stu_train_loader, stu_test_loader, stu_input_ids, stu_attention_masks,␣
 ↪stu_label_ids = task_3_solution()

# Check return types
```

```
assert isinstance(stu_train_loader, DataLoader), f"Task {task_id}: The returned␣
 ↪`train_loader` should be a PyTorch DataLoader."
assert isinstance(stu_test_loader, DataLoader), f"Task {task_id}: The returned␣
 ↪`test_loader` should be a PyTorch DataLoader."
assert isinstance(stu_input_ids, torch.Tensor), f"Task {task_id}: The returned␣
 ↪`input_ids` should be a torch.Tensor."
assert isinstance(stu_attention_masks, torch.Tensor), f"Task {task_id}: The␣
 ↪returned `attention_masks` should be a torch.Tensor."
assert isinstance(stu_label_ids, torch.Tensor), f"Task {task_id}: The returned␣
 ↪`label_ids` should be a torch.Tensor."
```

```
Task 2 - AG tests
```

5 Task 4: Build Model

Here, we're going to define the model and optimizer we will be using for our transformer, which will be returned by the function below.

Set the model as `BertForTokenClassification.from_pretrained()`. The first argument should be to the model we've stored for you in assets/bert_token_classification, and num_labels should be 4.

Optimizer should be `AdamW()`, with the first argument being `model.parameters()` and the learning rate `lr` argument set to the number `5e-5`.

[29]:
```
task_id = '4'

## Utility Code will go here
```

[30]:
```
def task_4_solution():
    # Load model
    model = BertForTokenClassification.from_pretrained('assets/
 ↪bert_token_classification', num_labels = 4)

    # Optimizer
    optimizer = AdamW(model.parameters(), lr = 5e-5)

    # YOUR CODE HERE
    #raise NotImplementedError()

    return model, optimizer
```

[31]:
```
# Use this cell to explore your solution.

task_4_solution()
```

```
[31]: (BertForTokenClassification(
        (bert): BertModel(
          (embeddings): BertEmbeddings(
            (word_embeddings): Embedding(30522, 768, padding_idx=0)
            (position_embeddings): Embedding(512, 768)
            (token_type_embeddings): Embedding(2, 768)
            (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
            (dropout): Dropout(p=0.1, inplace=False)
          )
          (encoder): BertEncoder(
            (layer): ModuleList(
              (0-11): 12 x BertLayer(
                (attention): BertAttention(
                  (self): BertSdpaSelfAttention(
                    (query): Linear(in_features=768, out_features=768, bias=True)
                    (key): Linear(in_features=768, out_features=768, bias=True)
                    (value): Linear(in_features=768, out_features=768, bias=True)
                    (dropout): Dropout(p=0.1, inplace=False)
                  )
                  (output): BertSelfOutput(
                    (dense): Linear(in_features=768, out_features=768, bias=True)
                    (LayerNorm): LayerNorm((768,), eps=1e-12,
      elementwise_affine=True)
                    (dropout): Dropout(p=0.1, inplace=False)
                  )
                )
                (intermediate): BertIntermediate(
                  (dense): Linear(in_features=768, out_features=3072, bias=True)
                  (intermediate_act_fn): GELUActivation()
                )
                (output): BertOutput(
                  (dense): Linear(in_features=3072, out_features=768, bias=True)
                  (LayerNorm): LayerNorm((768,), eps=1e-12, elementwise_affine=True)
                  (dropout): Dropout(p=0.1, inplace=False)
                )
              )
            )
          )
        )
        (dropout): Dropout(p=0.1, inplace=False)
        (classifier): Linear(in_features=768, out_features=4, bias=True)
      ),
      AdamW (
      Parameter Group 0
          betas: (0.9, 0.999)
          correct_bias: True
          eps: 1e-06
```

```
    lr: 5e-05
    weight_decay: 0.0
))
```

[32]:
```
print(f"Task {task_id} - AG tests")

# Call the student solution
stu_model, stu_optimizer = task_4_solution()

# Check return types
assert isinstance(stu_model, BertForTokenClassification), f"Task {task_id}: The␣
 ↪returned `model` should be an instance of `BertForTokenClassification`."
assert isinstance(stu_optimizer, AdamW), f"Task {task_id}: The returned␣
 ↪`optimizer` should be an instance of `AdamW`."

# Check model configuration
assert stu_model.config.num_labels == 4, f"Task {task_id}: The `num_labels` in␣
 ↪the model configuration should be 4. Found {stu_model.config.num_labels}."
assert stu_model.config._name_or_path == "assets/bert_token_classification",␣
 ↪f"Task {task_id}: The model should use the pretrained `assets/
 ↪bert_token_classification`. Found {stu_model.config._name_or_path}."

# Check optimizer configuration
optimizer_params = list(stu_optimizer.param_groups[0].keys())
assert "lr" in optimizer_params, f"Task {task_id}: The optimizer should have a␣
 ↪`lr` parameter."
assert stu_optimizer.param_groups[0]["lr"] == 5e-5, f"Task {task_id}: The␣
 ↪learning rate for the optimizer should be `5e-5`. Found {stu_optimizer.
 ↪param_groups[0]['lr']}."
```

```
Task 4 - AG tests
```

6 Task 5: Evaluate model

Now, you've conducted the steps necessary to use the transformer to make predictions on the labels of the address dataset.

The code itself to actually train and simulate the model is similar to the code we've written in the previous module with the Bidirectional-LSTM CRF model. However, this is too computationally expensive to execute on our systems, so we have prepared the outputs of a trained model for you in a precomputed file. Your job here is simply to deduce the accuracy of the outputs and return it in the function below.

To do this,

We Loop through each batch in `precomputed_predictions`. For each batch you can access "predictions" and "label_ids" of the batch with something like `batch["predictions"]`. We've done all this and converted these to numpy 2d arrays for you.

Now, your job is to loop through these arrays and find the accuracy of the transformer model.

- Hint: Padding tokens are stored in `label_ids` ndarrays as the integer `-100`. Make sure to exclude padding when computing accuracy. For example, if we have

preds = [[0, 0], [0, 0]]

label_ids = [[-100, 1, 0, -100], [-100, 0, 0, -100]]

Accuracy here would be 75%.

[33]:
```
task_id = '5'

## Utility Code will go here
```

[58]:
```
def task_5_solution():
    # Load precomputed predictions
    precomputed_predictions = torch.load("assets/precomputed_predictions.pt")

    accuracy = 0



    correct, total = 0, 0

    # Evaluate predictions
    for batch in precomputed_predictions:
        predictions = batch["predictions"].numpy()
        label_ids = batch["label_ids"].numpy()

        # YOUR CODE HERE
        #raise NotImplementedError()
        for i in range(len(predictions)):
            preds_ = predictions[i][label_ids[i] != -100]
            label_ids_ = label_ids[i][label_ids[i] != -100]
            #print(preds_)
            #print(label_ids_)
            correct += sum(pred == true for pred, true in zip(preds_,␣
 ↪label_ids_))
            total += len(preds_)

    accuracy = correct / total if total > 0 else 0
    return accuracy

    return accuracy
```

[59]:
```
# Use this cell to explore your solution.
```

```
task_5_solution()
```

[59]:
```
0.9918533604887984
```

[60]:
```
print(f"Task {task_id} - AG tests")

# Call the student solution
stu_accuracy = task_5_solution()

# Check return type
assert isinstance(stu_accuracy, float), f"Task {task_id}: The returned␣
 ↪`accuracy` should be of type `float`. Found {type(stu_accuracy)}."
```

```
Task 6 - AG tests
```

7 Task 6

Now, you've computed the accuracy of predicting address fields with CRF, Bidirectional LSTM-CRF, and a Transformer Model. There should be a slight performance difference between each model. In this task, we'd like you to rank the accuracy of the 3 prediction methods you used.

Using the string variables we've defined for you below, rank the each method in ascending order of accuracy.

Note: The accuracy you obtain from these models can vary from run to run. Models that are more accurate than others may have an unlucky run and come out with a lower accuracy than a less accurate model. If you've ranked them based on a single run of the models and came out with the incorrect answer, try running the models multiple times and rank them based on the average values you see.

[61]:
```
task_id = '6'

## Utility Code will go here
```

[62]:
```
def task_6_solution():
    BidirectionalLSTMCRF = "BidirectionalLSTMCRF"
    crf = "crf"
    Transformer = "Transformer"

    sol = [crf, BidirectionalLSTMCRF, Transformer]
    # YOUR CODE HERE
    #raise NotImplementedError()

    return sol
```

[63]:
```
# Use this cell to explore your solution.

task_6_solution()
```

[63]:
```
['crf', 'BidirectionalLSTMCRF', 'Transformer']
```

[64]:
```
print(f"Task {task_id} - AG tests")
stu_ans = task_6_solution()

print(f"Task {task_id} - your answer:\n{stu_ans}")

assert isinstance(
    stu_ans, list
), f"Task {task_id}: Your function should return a list. "

assert (
    len(stu_ans) == 3
), f"Task {task_id}: Your returned list has an incorrect number of answers. "


assert all(
    isinstance(ans, str) for ans in stu_ans
), (
    f"Task {task_id}: Your returned list should be a list of strs"
)
```

```
Task 6 - AG tests
Task 6 - your answer:
['crf', 'BidirectionalLSTMCRF', 'Transformer']
```

[ ]: